

Technical & Strategic Execution Plan: Fruitcyclopedia (formerly Fruition)

To: The Executive Team

From: Chief Product Officer & Technical Lead

Date: December 21, 2025

Subject: The Pivot to Intelligence: Strategy, Compliance, and Technical Architecture for the "Tracker-First" Era

Executive Summary: The Intelligent Pivot

The strategic realignment of "Fruitcyclopedia" from a passive reference library to an active, habit-forming utility represents the single most critical evolution in our product's lifecycle. Our previous trajectory—a "content-heavy, engagement-light" encyclopedia—carried significant retention risks and, more critically, existential threats regarding Apple App Store approval. By shifting the "Tracker" to the position of the daily driver and subordinating the "Encyclopedia" to a backend intelligence layer, we not only solve the engagement problem but also construct a robust defense against the "Wrapper" classification that plagues hybrid applications.

This report serves as the comprehensive tactical roadmap for this pivot. It is exhaustive in its technical detail and strategic foresight, designed to guide engineering, product, and legal teams through the complexities of launching a high-performance health startup in a regulated and highly competitive ecosystem. We are not merely building an app; we are constructing a "Smart Companion" that leverages on-device machine learning, authoritative government data, and generative 3D assets to create a user experience that feels natively "alive."

The analysis that follows is divided into four primary strategic pillars:

1. **The Wrapper Defense:** A technical architecture designed to leverage local persistence (SQLite) and native hardware bridges to satisfy Apple's strict Guideline 4.2 requirements.
2. **The Data Automation Pipeline:** A scientifically rigorous ingestion strategy using USDA Foundation Foods to provide "The Authority" without manual overhead.
3. **The Intelligent Tracker:** A detailed implementation plan for "Zero-Friction" logging using Google ML Kit and generative 3D visualization via Meshy.ai.
4. **Brand & Legal Compliance:** A framework for securing our intellectual property and insulating the company from medical liability.

1. The "Wrapper" Defense: Engineering for App Store

Survival

The most immediate and lethal risk to the Fruitcyclopedia launch is rejection by the Apple App Store Review Board under **Guideline 4.2 (Minimum Functionality)**. Our current build pipeline—converting a web-based codebase into an Xcode project via "Google Antigravity" (referencing our web-to-native transformation workflow)—inherently produces a "Hybrid" application. Historically, Apple has viewed such applications with skepticism, frequently categorizing them as "repackaged websites" unworthy of the App Store ecosystem.¹

To secure approval, we must move beyond simply "wrapping" our web content. We must implement a **"Hybrid Native" architecture** that forces the application to behave, perform, and interact with the operating system indistinguishably from a native Swift or Kotlin application. This requires a fundamental shift in how we handle data, navigation, and hardware integration.

1.1 Deconstructing the Threat: Guidelines 4.2 and 4.7

Understanding the specific mechanics of Apple's rejection criteria is the first step in our defense. The guidelines are not merely rules; they are quality filters designed to curate the App Store as a premium software repository, distinct from the open web.

1.1.1 Guideline 4.2: The "Minimum Functionality" Threshold

Guideline 4.2 states: *"Your app should include features, content, and UI that elevate it beyond a repackaged website. If your app is not particularly useful, unique, or 'app-like,' it doesn't belong on the App Store."*¹

The term "App-like" is the operational keyword here. In the eyes of a reviewer, an app ceases to be "App-like" when:

- **Latency exists in navigation:** If a user taps a button and waits for a server round-trip to render the next screen, the app feels like a browser. Native apps render the UI immediately, even if the data is still loading.
- **Network dependency is absolute:** If the app displays a "404 Error" or a generic "No Internet" message when distinct from the web, it fails the utility test. A native app should provide value regardless of connectivity.³
- **UI Conventions are web-based:** The presence of small touch targets, "footer" links, or non-standard navigation stacks signals a web wrapper.

The "Wrapper" trap effectively kills startups that assume they can simply point a WKWebView to a URL. Apple's reviewers actively reject apps that do not utilize the device's capabilities to provide an experience superior to Safari.⁴

1.1.2 Guideline 4.7: The Limits of Web Embeds

Guideline 4.7 governs "Mini apps, mini games, streaming games, chatbots, plug-ins, and game emulators".² While this guideline permits the embedding of HTML5 content—essential for our "Encyclopedia" views—it strictly regulates the interaction between that content and the native shell. It reinforces that the *container* (the Fruitcyclopedia app) must be the primary driver of value, not just a window into a remote server. We must ensure that our tracking logic, camera integrations, and core navigation reside in the native layer (or a locally hosted web layer) rather than being fetched remotely at runtime.

1.2 The "Hybrid Native" Architecture: The SQLite Solution

The single most effective technical defense against a Guideline 4.2 rejection is the implementation of an **Offline-First Data Architecture**. By decoupling the application's user interface from the network state, we prove to the reviewer that Fruitcyclopedia is a standalone software artifact.

1.2.1 The Capacitor SQLite Strategy

We will utilize **Capacitor**, specifically the @capacitor-community/sqlite plugin, to manage a fully relational database directly on the user's device.⁵

The Architectural Shift:

- **Current State (Risky):** The app requests data from api.fruitcyclopedia.com. If the internet is slow, the app hangs. If the internet is off, the app fails.
- **Target State (Defensible):**
 1. **Local Persistence:** The "Authority" database (USDA nutrient data) and the user's "Tracker" logs are stored in a local SQLite database file on the iOS file system.
 2. **Instant Read:** When the user opens the "Tracker" tab, the app queries the local SQLite database. This query takes milliseconds, allowing the UI to populate instantly.⁵
 3. **Background Sync:** A background process (using a sync engine like PowerSync or a custom delta-sync implementation) synchronizes the local changes with the cloud backend *after* the UI has rendered.⁷

Implementation Detail:

The Capacitor SQLite plugin interfaces directly with the native SQLite engine on iOS and Android. This bypasses the browser's IndexedDB or LocalStorage, which are unreliable and can be wiped by the operating system under low-storage conditions.⁵ This permanence is a key differentiator between a "website" and an "app."

Reviewer Test Scenario:

When the Apple Reviewer installs Fruitcyclopedia, one of their first actions will likely be to activate "Airplane Mode" and attempt to use the app.

- **Fail Scenario:** The app shows a white screen or a "Connection Error." -> **Rejection (Guideline 4.2).**
- **Pass Scenario:** The app opens, the dashboard loads user history from the local DB, the "Scan Fruit" button works (using on-device ML), and the user can log a meal (queued for

later sync). -> **Approval.**

1.2.2 Native Performance Optimization

To further distance the app from its web origins, we must eliminate the "Webview Lag."

- **No Full-Page Spinners:** We must banish the "loading spinner" that blocks the entire screen. Instead, we will use "Skeleton Screens"—gray placeholders that mimic the layout of the text/images—which are instantly replaced by data from the SQLite cache. This creates the illusion of zero latency.
- **Touch Response:** Webviews often have a 300ms delay on tap events. We must use CSS properties like touch-action: manipulation and FastClick libraries to ensure buttons respond instantly to input.

1.3 The "Defense Trinity": Mandatory Native Integrations

Beyond offline capability, we must integrate features that strictly require native APIs. We will implement three specific features that a standard mobile website in Safari cannot easily replicate. These serve as our "proof of life" to the App Store Review Board.

1.3.1 Feature 1: Push Notifications (The Habit Loop)

While "Web Push" exists, it is functionally distinct from **Apple Push Notification Service (APNS)**. Implementing native push notifications demonstrates deep system integration.³

- **Strategic Utility:** The "Tracker" relies on habit formation. We will implement "Smart Reminders" (e.g., "You haven't logged a fruit today!") that deep-link directly to the logging screen.
- **Compliance Value:** Reviewers view Push Notifications as a quintessential "App" feature. It proves we are using the app to re-engage users, something a passive website struggles to do effectively.

1.3.2 Feature 2: Native Camera Access (The "Intelligent" Eye)

We cannot simply use an <input type="file"> tag, which opens the generic system photo picker. This is a "web" pattern.

- **Strategic Utility:** We must build a **Custom Camera UI** within the app using the capacitor-camera-preview or similar plugins. This allows us to overlay the ML recognition bounding boxes directly onto the live camera feed.
- **Compliance Value:** This utilizes the device's hardware (GPU and Camera sensors) continuously. It requires the NSCameraUsageDescription in the Info.plist, explicitly signaling to Apple that this app is a hardware-integrated tool, not a content browser.³

1.3.3 Feature 3: Haptic Feedback (The Tactile Bridge)

The subtle vibration of a phone when a user interacts with UI elements is a powerful

subconscious signal of "native quality."

- **Strategic Utility:** When a user successfully logs a fruit, the phone should emit a "Success" vibration pattern. When they scroll through the "Fruitcyclopedia" list, the "tick-tick" of a selection wheel adds weight to the interaction.
- **Compliance Value:** Webpages generally do not access the Taptic Engine. Using the Haptics Capacitor plugin allows us to trigger `Haptics.impact({ style: ImpactStyle.Light })` on interactions. This is a low-effort, high-impact signal of "App-ness".⁹

1.4 Execution Checklist for Xcode Submission

To ensure the build passing through "Google Antigravity" lands in Xcode ready for approval, the following configuration steps are mandatory:

1. **Info.plist Configuration:**
 - `NSCameraUsageDescription`: *"Fruitcyclopedia needs camera access to scan and identify fruits for your daily log."*
 - `NSPhotoLibraryUsageDescription`: *"We need access to your photos to let you import past meals into your tracker."*
 - `NSLocationWhenInUseUsageDescription` (Optional but recommended): *"We use your location to suggest locally in-season fruits."*
2. **App Review Notes:**
 - We must explicitly guide the reviewer. In App Store Connect, under "App Review Information > Notes," we will write: *"This application is designed as a hybrid-native utility. It utilizes local SQLite databases for full offline functionality, the native camera for on-device Machine Learning image recognition, and native push notifications for habit tracking. It is not a web wrapper; it is a standalone tool powered by a local intelligence engine."*¹⁰
3. **OAuth Compliance:**
 - If we offer "Sign in with Google," we **must** offer "Sign in with Apple" to comply with Guideline 4.8. The Apple Sign-In flow must be native, not a web redirect.²

2. The Data Automation Pipeline: Building "The Authority"

With the "Tracker" established as the daily driver, "The Authority" (the encyclopedia) becomes the silent intelligence engine. Manual data entry for thousands of fruit varieties is a non-starter; it is error-prone, unscalable, and legally risky. We require an automated pipeline to ingest, normalize, and serve nutritional data.

2.1 Comparative Analysis: USDA FoodData Central vs. OpenFoodFacts

We evaluated the two primary global food databases: the **USDA FoodData Central (FDC)**

and **OpenFoodFacts (OFF)**.

Feature Category	USDA FoodData Central (FDC)	OpenFoodFacts (OFF)	Analysis for Fruitcyclopedia
Data Provenance	Gold Standard. Data is derived from rigorous scientific sampling (Foundation Foods) or industry reporting (Branded).	Crowdsourced. Data is submitted by users (like Wikipedia). High variability in quality and completeness.	USDA Wins. For a health startup, accuracy is liability. We cannot risk advising a diabetic based on user-edited sugar values. ¹¹
Target Domain	Raw Commodities. Excellent coverage of fruits, vegetables, and basic ingredients via "Foundation Foods."	Packaged Goods. Excellent coverage of barcoded products (Oreos, frozen pizza). Poor coverage of raw produce.	USDA Wins. Our core value proposition is <i>fruit</i> (raw commodities). OFF is better for a "Snack Tracker". ¹¹
Micronutrient Depth	Exhaustive. Includes 150+ nutrients (Vitamins, Minerals, Amino Acids). ¹³	Shallow. Often limited to the "Nutrition Facts" label (Macros: Fat, Carbs, Protein).	USDA Wins. "Intelligent" tracking means telling the user they hit their <i>Vitamin C</i> goal, not just their calorie goal.
Commercial License	Public Domain (CC0). U.S. Government work. Free for commercial use without attribution or "share-alike" clauses.	ODbL (Open Database License). "Share-Alike" requirement. If we enrich their data, we may be forced to open-source our database.	USDA Wins. We need to own our proprietary, enriched dataset without legal encumbrances. ¹³

Strategic Decision: We will build our pipeline primarily on the **USDA FoodData Central API**, specifically targeting the **Foundation Foods** data type.

2.2 The "Foundation Foods" Ingestion Strategy

The USDA API provides multiple data types. "SR Legacy" is the old standard, but "Foundation Foods" is the modern, highly granular dataset we need.

The Data Model Challenge:

USDA data is normalized to 100 grams. Users, however, do not eat "100 grams of apple"; they eat "one medium apple." The bridge between these two units is the critical algorithmic challenge of the pipeline.

2.2.1 The Ingestion Algorithm

We will not query the USDA API live from the user's device (latency/rate limit risk). Instead, we will implement a backend **ETL (Extract, Transform, Load) Pipeline**:

1. **Extract:**

- Script queries <https://api.nal.usda.gov/fdc/v1/foods/list> filtering for `dataType=Foundation` and `categoryId` corresponding to "Fruits and Fruit Juices".¹⁴
- Example Target: "Apples, fuji, with skin, raw" (FDC ID: 171689).¹⁵

2. **Transform (The Gram-Weight Logic):**

- **Nutrient Mapping:** Extract key nutrients using their NDB IDs (e.g., Vitamin C = 401, Energy = 208, Sugar = 269).¹⁶
- **Portion Conversion:** We must parse the `foodPortions` array in the JSON response.
 - We look for a `measureUnit` description that matches user mental models (e.g., "medium", "large", "cup").
 - We extract the `gramWeight` field associated with that measure.¹⁷
 - *Example:* The API returns a portion description "1 medium (3" dia)" with a `gramWeight` of **182g**.

3. **Load:**

- This transformed data is loaded into our **Fruitcyclopedia Authority Database** (SQLite format).
- This database file is bundled with the app update, ensuring the "Encyclopedia" is available offline.

2.2.2 The Math of "Intelligent" Tracking

When a user logs "1 Medium Fuji Apple," the app performs the following calculation locally:

$$\text{UserNutrient} = \left(\frac{\text{PortionWeight}_{\text{g}}}{100} \right) \times \text{USDA NutrientValue}_{\text{per100g}}$$

Example:

- **User Input:** "1 Medium Fuji Apple"

- **Data:** Portion Weight = 182g; Vitamin C (per 100g) = 4.6mg.
- **Calculation:** $(182 / 100) \times 4.6 = 8.37 \text{mg}$
This level of precision—calculating micronutrients based on specific varietal weights—is the "Intelligence" that differentiates Fruitycyclopedia from generic trackers that just say "Apple: 95 calories."

2.3 Handling Data Gaps

While USDA is excellent, it may miss obscure exotic fruits (e.g., Mangosteen varieties).

- **Fallback Strategy:** We will create a secondary "Scraping Pipeline" for missing items, potentially using LLM-verified data from agricultural university extensions, flagged in the database as "Estimated" to protect our liability.

3. The "Intelligent" Tracker: Vision and Visualization

The "Tracker" must be the daily driver. To achieve this, we must reduce the friction of logging to near zero. A text-based search ("Type 'Apple' -> Select 'Apple'") is high friction. A visual scan ("Point Camera -> Confirm 'Apple'") is zero friction.

3.1 Computer Vision: Google ML Kit vs. Nyckel

We compared **Google ML Kit** (On-Device) against **Nyckel** (Cloud API) for the fruit recognition engine.

Feature	Google ML Kit (Vision API)	Nyckel	Verdict for MVP
Architecture	On-Device. Inference happens locally on the phone's Neural Engine/DSP.	Cloud-Based. Images are uploaded to Nyckel servers for processing.	ML Kit Wins. On-device is faster and privacy-compliant (no fruit photos leave the phone). ¹⁸
Latency	Real-time (<100ms). Capable of analyzing live video frames.	Network Dependent (~1-2s). Requires upload/download round trip.	ML Kit Wins. Instant feedback is crucial for a "magic" feel.

Cost	Free. Included in the Firebase/Google SDKs.	Paid. Tiered pricing per 1,000 images. Recurring cost scales with user growth.	ML Kit Wins. Zero marginal cost per user.
Accuracy	Generalist. The default model recognizes ~400+ common objects, including generic "Fruit," "Apple," "Banana," "Lemon". ²⁰	Specialist. Can be custom trained to distinguish specific varieties (Honeycrisp vs. Gala) with high accuracy.	ML Kit Wins (Phase 1). The default labels are sufficient for an MVP. ²⁰

3.1.1 The ML Kit Implementation Strategy

We will use the **Google ML Kit Image Labeling API** with the default on-device model.

- **Labels:** The base model already supports labels like Fruit, Apple, Banana, Strawberry, Orange, Lemon, Pear, Grape.²⁰
- **Workflow:**
 1. User taps "Scan".
 2. capacitor-camera-preview opens a live view.
 3. Frames are passed to the ML Kit plugin.
 4. When confidence for a "Fruit" label exceeds 75%, a "chip" appears on screen: *"It looks like an Apple. Log it?"*
 5. **Phase 2 Upgrade:** If users demand varietal detection (e.g., "Is this a Fuji?"), we will train a custom **TensorFlow Lite (TFLite)** model using a dataset of specific apple varieties and bundle it with the app. This allows us to keep the inference *on-device* while upgrading the intelligence, retaining the privacy and speed benefits of ML Kit.²²

3.2 The 3D Pipeline: Meshy.ai Integration

To make the "Encyclopedia" visually distinct and "premium," we will replace static 2D photos with rotatable 3D models. This utilizes the **Meshy.ai** generative AI platform.

3.2.1 The Generative Workflow

We will not generate models in real-time (latency/cost is too high). We will pre-generate a "Core Collection" of the top 50 fruits.

1. **Generation:** Use Meshy.ai's Image to 3D API.
 - **Input:** High-resolution photo of a "Red Delicious Apple."
 - **Cost:** "Meshy-6" models cost **20 credits** per generation. With the **Pro Plan**

(\$20/mo), we receive 1,000 credits/month.²⁴ This allows us to generate ~50 high-quality assets per month, sufficient to build our library incrementally.

- **Format:** Request .glb (binary glTF), which is the standard for web and mobile rendering.

2. Asset Management:

- These .glb files (approx 2-5MB each) are hosted on our CDN.
- When the app installs or updates, the most popular models are cached to the device's local filesystem (using capacitor-filesystem). This ensures the 3D viewer works **offline**, reinforcing the "Wrapper Defense."

3.3 Rendering Engine: vs. Three.js

For displaying these 3D models, we have two choices: (a Google web component) or **Three.js** (a raw WebGL library).

Feature		Three.js	Verdict
Ease of Integration	Drop-in. <model-viewer src="apple.glb" ar> handles loading, lighting, and interaction automatically.	High Complexity. Requires setting up scenes, cameras, lighting, and render loops manually.	Wins. drastically reduces development time. ²⁶
AR Capabilities	Native Support. Built-in "View in AR" button leverages WebXR and iOS QuickLook.	Manual. Requires complex implementation to trigger AR sessions.	Wins. The AR feature is a huge "native feel" booster.
Performance	Optimized. Handles memory management and progressive loading automatically.	Developer-Dependent. Easy to cause memory leaks in a React/Capacitor environment if not managed perfectly.	Wins. Stability is key.
Offline Loading	Tricky. Requires correct local path	Flexible. Can load from any	Wins (with config).

	configuration (CORS issues with local files). ²⁷	buffer/blob.	
--	---	--------------	--

Implementation Decision: We will use .

To solve the "Offline" issue 27, we will configure the Capacitor server to serve local files from a specific directory (e.g., `_capacitor_file_`) and pass that local URI to the `src` attribute of the `<model-viewer>` component. This allows the user to rotate a 3D apple even when in Airplane Mode.

4. Brand, Legal & Compliance Framework

The transition to a "Smart Health" tool introduces new liabilities. We must secure our brand and protect ourselves from the risks of providing "health information."

4.1 Trademark Strategy: "Fruitcyclopedia"

A preliminary analysis of the USPTO database suggests "Fruitcyclopedia" is a strong candidate for registration.²⁸

- **Distinctiveness:** While "Fruit" is descriptive (weak), the suffix "-cyclopedia" combined with it creates a "Portmanteau," which is generally considered "Suggestive" rather than "Descriptive." Suggestive marks are easier to protect.
- **Action Plan:**
 1. **Knockout Search:** verify no phonetic equivalents exist (e.g., "Fruito-pedia").
 2. **Filing Classes:** We must file in:
 - **Class 9:** Downloadable Mobile Applications (Software).
 - **Class 42:** Software as a Service (SaaS) (Backend data services).
 - **Class 44:** Provision of information relating to nutrition and health.³⁰

4.2 Medical Liability & The Disclaimer

Because we calculate micronutrients (e.g., "This apple has 12g of sugar"), a diabetic user might rely on this data. If the data is wrong, we could be sued. We must explicitly disclaim this liability.

The Medical Disclaimer (Draft Text):

*"The Content provided by Fruitcyclopedia, including but not limited to nutrient data, 3D visualizations, and serving size estimations, is for **informational and educational purposes only** and is not intended to substitute for professional medical advice, diagnosis, or treatment. Fruitcyclopedia is not a medical device. Always seek the advice of your physician or other qualified health provider with any*

*questions you may have regarding a medical condition or dietary changes. Reliance on any information provided by this Application is solely at your own risk. We do not guarantee the accuracy of nutritional data, which is sourced from third-party databases."*³¹

Placement: This text must be presented during the **Onboarding Flow** with a mandatory "I Agree" checkbox. It should also be permanently accessible in the **Settings > Legal** menu.

4.3 Privacy Policy & Health Research

Apple's **Guideline 5.1.3** is extremely strict regarding health data. We are collecting information about what users eat, which serves as a proxy for their health.²

Privacy by Design Requirements:

1. **Data Minimization:** We must state that food logs are processed and stored locally (via SQLite) and are only synced to the cloud for backup/multi-device support.
2. **No Advertising:** We must explicitly state: *"We do not sell, rent, or share your personal health data or food logs with third-party advertisers."* This is a requirement for apps that integrate with Apple Health (HealthKit).³⁵
3. **User-Generated Content (Food Photos):** Since users capture photos for the ML Tracker, our policy must address this: *"Photos captured via the Fruitycyclopedia Tracker are processed locally on your device for the purpose of fruit recognition. These images are not uploaded to our servers unless you explicitly choose to share them or save them to your cloud history."*³⁷

5. Tactical Launch Roadmap

To execute this pivot effectively, the development should follow this phased roadmap.

Phase 1: The "Iron Core" (Weeks 1-4)

- **Goal:** Establish the Hybrid Native architecture.
- **Tech:** Initialize Capacitor project. Install @capacitor-community/sqlite. Build the local database schema (Table: UserLogs, Table: FoundationFoods).
- **Data:** Write the USDA ETL script. Ingest the top 100 fruits. Map gramWeight logic. Generate a fruit_data.db seed file.
- **Compliance:** Draft the Privacy Policy and Medical Disclaimer. File Trademark Class 9.

Phase 2: The "Intelligent Eye" (Weeks 5-8)

- **Goal:** Build the "Zero-Friction" Tracker.
- **Tech:** Implement the Camera UI with capacitor-camera-preview. Integrate Google ML Kit (Image Labeling).

- **UX:** Build the "Scan -> Confidence Chip -> Log" interaction flow. Add Haptic Feedback on successful recognition.
- **3D:** Generate the top 20 fruit models via Meshy.ai. Implement <model-viewer> with local serving configuration for offline support.

Phase 3: The "Wrapper Defense" & Launch (Weeks 9-12)

- **Goal:** Polish and Submit.
- **Feature:** Implement Native Push Notifications for "Breakfast Reminders."
- **Test:** Conduct the "Airplane Mode Test." Ensure the app allows full logging and browsing without data.
- **Submission:** Prepare the App Store Connect metadata. Paste the "Hybrid Native" defense note in the Reviewer Notes field. Submit.

This roadmap moves Fruitcyclopedia from a vulnerable position to a defensible, high-value utility. By combining the "silent authority" of USDA data with the "active intelligence" of on-device ML, we create a product that is not just compliant, but indispensable.

Signed,

Chief Product Officer & Technical Lead

Works cited

1. App Store Review Guidelines: Will Your Webview App Be Rejected? - MobiLoud, accessed December 21, 2025, <https://www.mobiloud.com/es/blog/app-store-review-guidelines-webview-wrapper>
2. App Review Guidelines - Apple Developer, accessed December 21, 2025, <https://developer.apple.com/app-store/review/guidelines/>
3. App Store Rejection 4.2 - Design: Minimum Functionality - (my first after 2 years of Ionic), accessed December 21, 2025, <https://forum.ionicframework.com/t/app-store-rejection-4-2-design-minimum-functionality-my-first-after-2-years-of-ionic/200908>
4. Guideline 4.2 - Design - Minimum F... | Apple Developer Forums, accessed December 21, 2025, <https://developer.apple.com/forums/thread/806726>
5. Introducing the PowerSync Capacitor SDK (Alpha), accessed December 21, 2025, <https://releases.powersync.com/announcements/introducing-our-native-capacitor-sdk>
6. sqlite/docs/Web-Usage.md at master · capacitor-community/sqlite - GitHub, accessed December 21, 2025, <https://github.com/capacitor-community/sqlite/blob/master/docs/Web-Usage.md>
7. Easily keep a backend database synced with in-app SQLite for offline-first/local-first Capacitor apps - Reddit, accessed December 21, 2025, https://www.reddit.com/r/capacitor/comments/1op1cna/easily_keep_a_backend_d

- [atabase_synced_with_inapp/](#)
8. Fruit Identification using Arduino and TensorFlow Lite Micro, accessed December 21, 2025,
<https://blog.tensorflow.org/2019/11/fruit-identification-using-arduino-and-tensorflow.html>
 9. Launched my first app to the AppStore using Capacitor JS built with Vanilla JS - Reddit, accessed December 21, 2025,
https://www.reddit.com/r/capacitor/comments/1nrzpia/launched_my_first_app_to_the_appstore_using/
 10. App Store Review Guidelines (2025): Checklist + Top Rejection Reasons - NextNative, accessed December 21, 2025,
<https://nextnative.dev/blog/app-store-review-guidelines>
 11. Open Food Facts – a food products database made by everyone, for everyone | Hacker News, accessed December 21, 2025,
<https://news.ycombinator.com/item?id=31898218>
 12. Integration of USDA FoodCentral data into OFF - Technology - Open Food Facts Forum, accessed December 21, 2025,
<https://forum.openfoodfacts.org/t/integration-of-usda-foodcentral-data-into-off/408>
 13. Top 8 Nutrition APIs for Meal Planning 2024 - Eat Fresh Tech, accessed December 21, 2025,
<https://www.eatfresh.tech/blog/top-8-nutrition-apis-for-meal-planning-2024>
 14. FoodData Central API Guide, accessed December 21, 2025,
<https://fdc.nal.usda.gov/api-guide>
 15. Food Search | USDA FoodData Central, accessed December 21, 2025,
<https://fdc.nal.usda.gov/food-search?query=apples>
 16. FoodData Central Foundation Foods Documentation, accessed December 21, 2025, https://fdc.nal.usda.gov/Foundation_Foods_Documentation
 17. FoodData Central Download Field Descriptions Download Field ..., accessed December 21, 2025,
https://fdc.nal.usda.gov/docs/Download_Field_Descriptions_Oct2020.pdf
 18. AutoML benchmark: Nyckel vs Google vs Huggingface | by Oscar Beijbom - Medium, accessed December 21, 2025,
<https://medium.com/@oscar.beijbom/automl-benchmark-nyckel-vs-google-vs-huggingface-1cc0649ef27d>
 19. Compare Google Cloud Vision AI vs. Nyckel in 2025 - Slashdot, accessed December 21, 2025,
<https://slashdot.org/software/comparison/Google-Cloud-Vision-AI-vs-Nyckel/>
 20. Image labeling | ML Kit - Google for Developers, accessed December 21, 2025,
<https://developers.google.com/ml-kit/vision/image-labeling>
 21. Image Labeling | ML Kit for Firebase - Google, accessed December 21, 2025,
<https://firebase.google.com/docs/ml-kit/label-images>
 22. Custom models with ML Kit - Google for Developers, accessed December 21, 2025, <https://developers.google.com/ml-kit/custom-models>
 23. Object detection and tracking | ML Kit - Google for Developers, accessed

- December 21, 2025, <https://developers.google.com/ml-kit/vision/object-detection>
24. Pricing - Meshy Docs, accessed December 21, 2025, <https://docs.meshy.ai/api/pricing>
 25. What are your prices and plans offered, and do you have monthly/annual plans?, accessed December 21, 2025, <https://help.meshy.ai/en/articles/12062933-what-are-your-prices-and-plans-offered-and-do-you-have-monthly-annual-plans>
 - 26.
 27. model-viewer doesn't load models offline ? #4036 - GitHub, accessed December 21, 2025, <https://github.com/google/model-viewer/discussions/4036>
 28. Search our trademark database - USPTO, accessed December 21, 2025, <https://www.uspto.gov/trademarks/search>
 29. Federal trademark searching - USPTO, accessed December 21, 2025, <https://www.uspto.gov/trademarks/search/federal-trademark-searching>
 30. Trademark Class Search Tool - Bullock Law, accessed December 21, 2025, <https://www.bullock.law/trademark-classes>
 31. Medical Disclaimer Template - TermsFeed, accessed December 21, 2025, <https://www.termsfeed.com/blog/medical-disclaimer-template/>
 32. Medical Disclaimer Examples - Termly, accessed December 21, 2025, <https://termly.io/resources/articles/medical-disclaimer-examples/>
 33. Sample Disclaimer Template & Guide [Free Download] - Termly, accessed December 21, 2025, <https://termly.io/resources/templates/disclaimer-template/>
 34. Updates to the App Store Review Guidelines - Latest News - Apple Developer, accessed December 21, 2025, <https://developer.apple.com/news/?id=06032019j>
 35. Google's New Policies for Health Apps - Privacy World, accessed December 21, 2025, <https://www.privacyworld.blog/2023/12/googles-new-policies-for-health-apps/>
 36. Privacy policies for iOS apps | Data privacy guidelines - Harper James Solicitors, accessed December 21, 2025, <https://harperjames.co.uk/article/apple-privacy-policy-apps/>
 37. Privacy Policy | Food Fotos - GitHub Pages, accessed December 21, 2025, https://autoniq-ai.github.io/food_foto/privacy/
 38. Privacy Policy - FoodView, accessed December 21, 2025, <https://foodview.app/docs/policies/privacy/>